



ML FOR SCIENCE · LECTURE 2

ML Foundations for Scientific Data

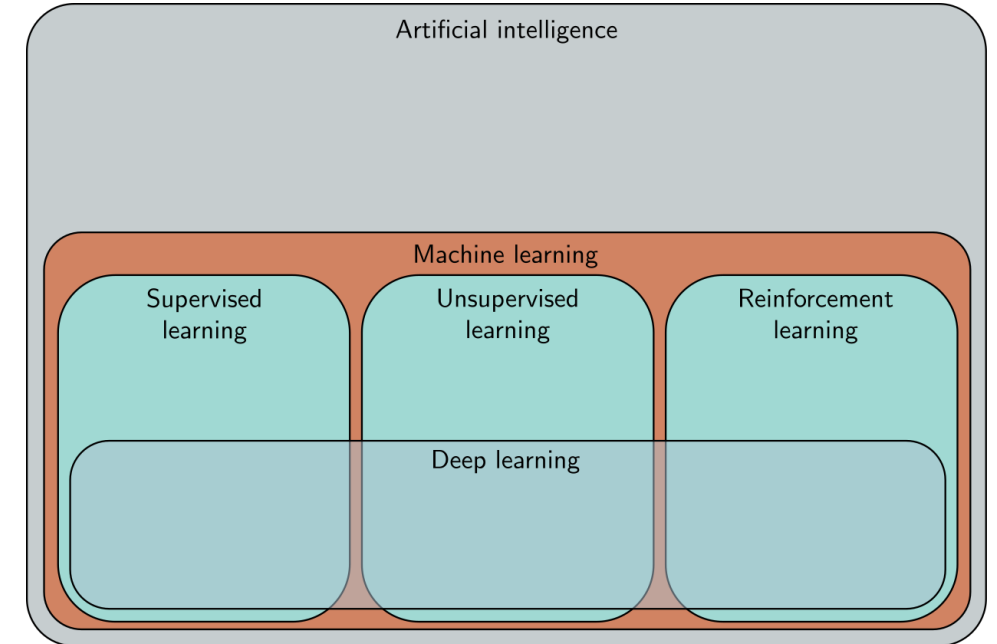
Recap: the Lecture 1 notebook

- **Data:** pairs of consecutive frames (u_n, u_{n+1})
- **Model:** a small **convolutional neural network**, SmallCNN
- **Loss:** **mean squared error** between prediction and target
- **Optimizer:** **Adam**, learning rate 10^{-3} , 10 epochs
- **Rollout:** feed each prediction back in as the next input

Today: a quick refresher on neural networks, training and generalization, then what changes when the data comes from physics.

Three types of machine learning

- **Supervised learning**
 - Learn a map from inputs to outputs from example pairs (x, y)
 - **SciML**: surrogate models, e.g. $u_n \rightarrow u_{n+1}$ in Lecture 1
- **Unsupervised learning**
 - Find structure in inputs alone, or learn to generate new samples
 - **SciML**: generative models of fields and materials
- **Reinforcement learning**
 - Learn actions that maximize a reward by interacting with an environment
 - **SciML**: magnetic control of tokamak plasmas (Degraeve et al., *Nature* 2022)



Deep learning (neural networks) is used in all three. **Self-supervised learning**, which trains foundation models, predicts hidden parts of the data itself.

Supervised learning

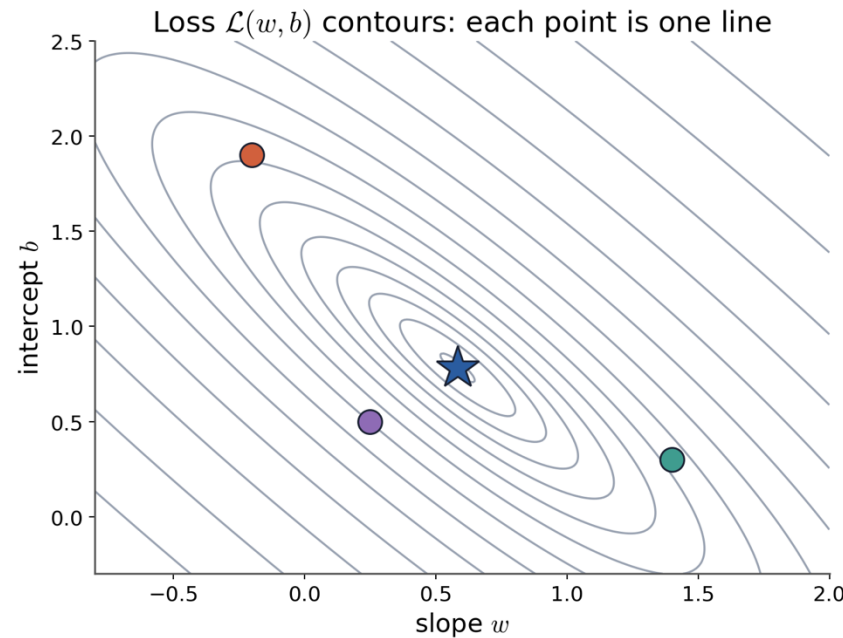
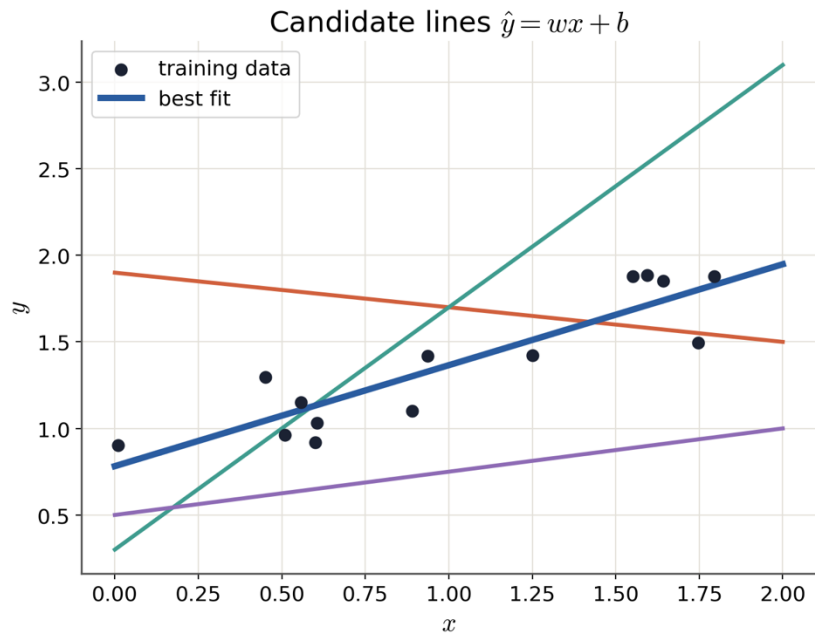
- **Training data:** N example pairs (x_i, y_i) , inputs with known outputs
- **Model:** a function $f(x; \theta)$, where θ collects all adjustable **parameters** (weights and biases)
- **Loss function** $\mathcal{L}(\theta)$: measures how far predictions are from targets
- **Training:** find the parameters that make the loss as small as possible

$$\hat{y} = f(x; \theta), \quad \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i; \theta), y_i), \quad \theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

- **Testing:** measure the error on data the model never saw during training
- **Regression** predicts **continuous values** (fields, temperatures)
- **Classification** predicts a **class label**

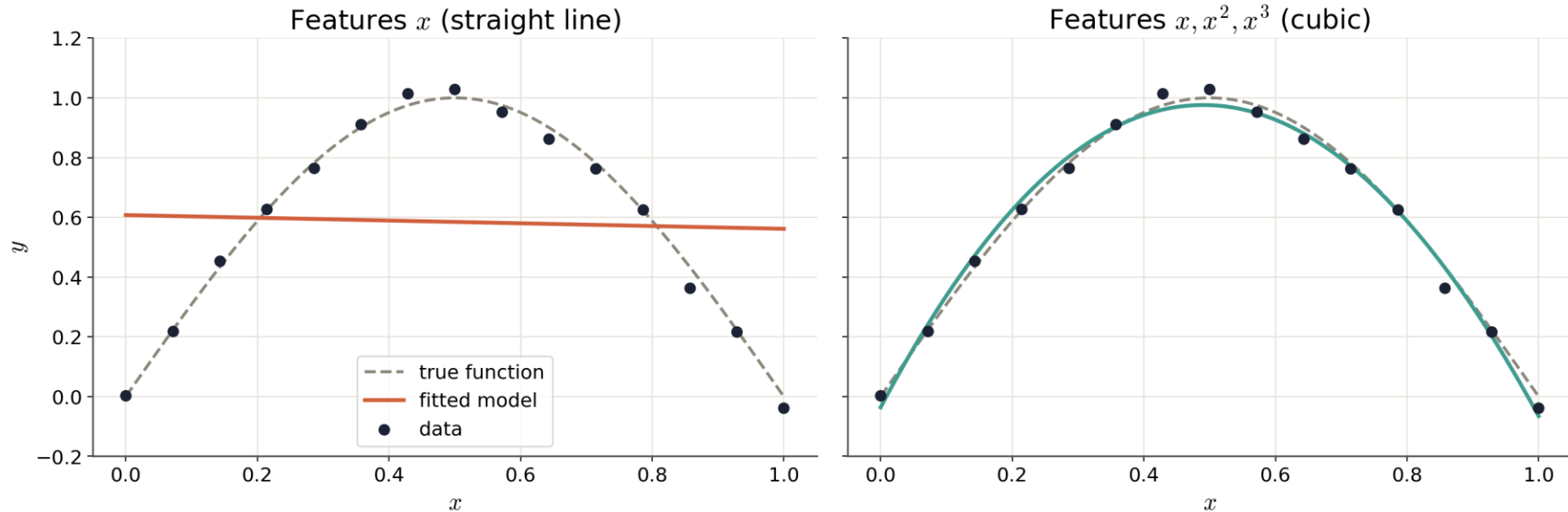
Simplest supervised model: linear regression

$$\hat{y} = w x + b, \quad \mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^N (w x_i + b - y_i)^2$$



- Two parameters: slope w , intercept b
- Each line is one point in (w, b) space
- Training = finding the lowest point (★)

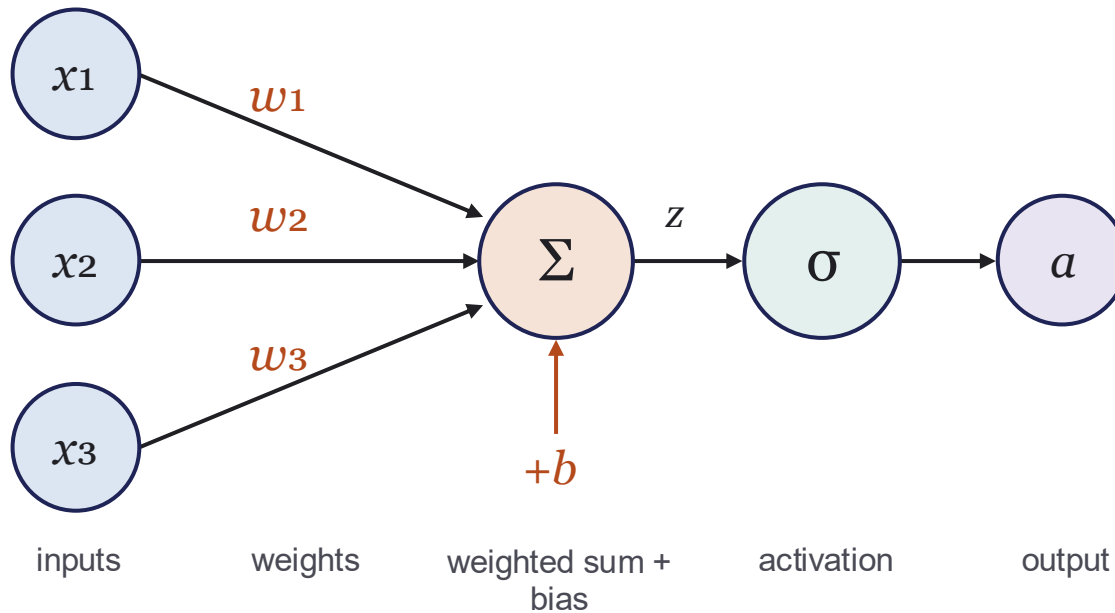
A straight line is not enough



- A model that is **linear in x** cannot follow **$\sin(\pi x)$**
- Linear regression on fixed nonlinear **features** ($x, x^2, x^3, \dots$) can, but you have to choose the features
- **Neural networks learn the features from data**

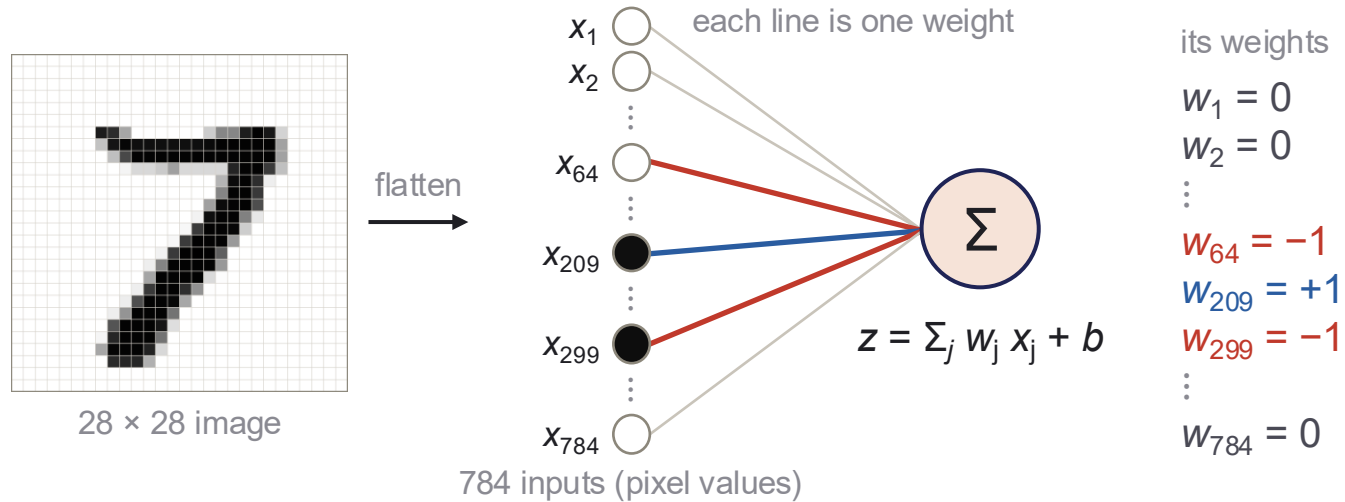
The artificial neuron (perceptron)

$$a = \sigma\left(\sum_{j=1}^n w_j x_j + b\right) = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$



- **Inputs** x_j , e.g. pixel values of an image
- **Weights** w_j : how much each input counts, and with which sign
- **Bias** b : shifts the sum, which sets how easily the neuron activates
- **Activation** σ : a fixed nonlinear function; a is the neuron's output
- **Weights** and **biases** are the parameters learned in training

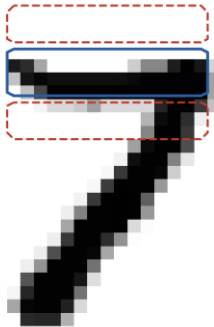
What weights and biases do



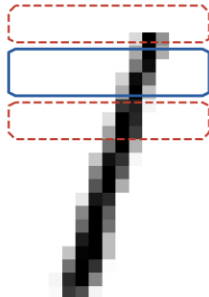
weights w_j of one neuron
blue +1, red -1, white 0



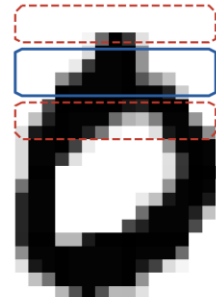
input: a "7"
 $\sum_j w_j x_j = 20.9$



input: a "1"
 $\sum_j w_j x_j = -1.0$

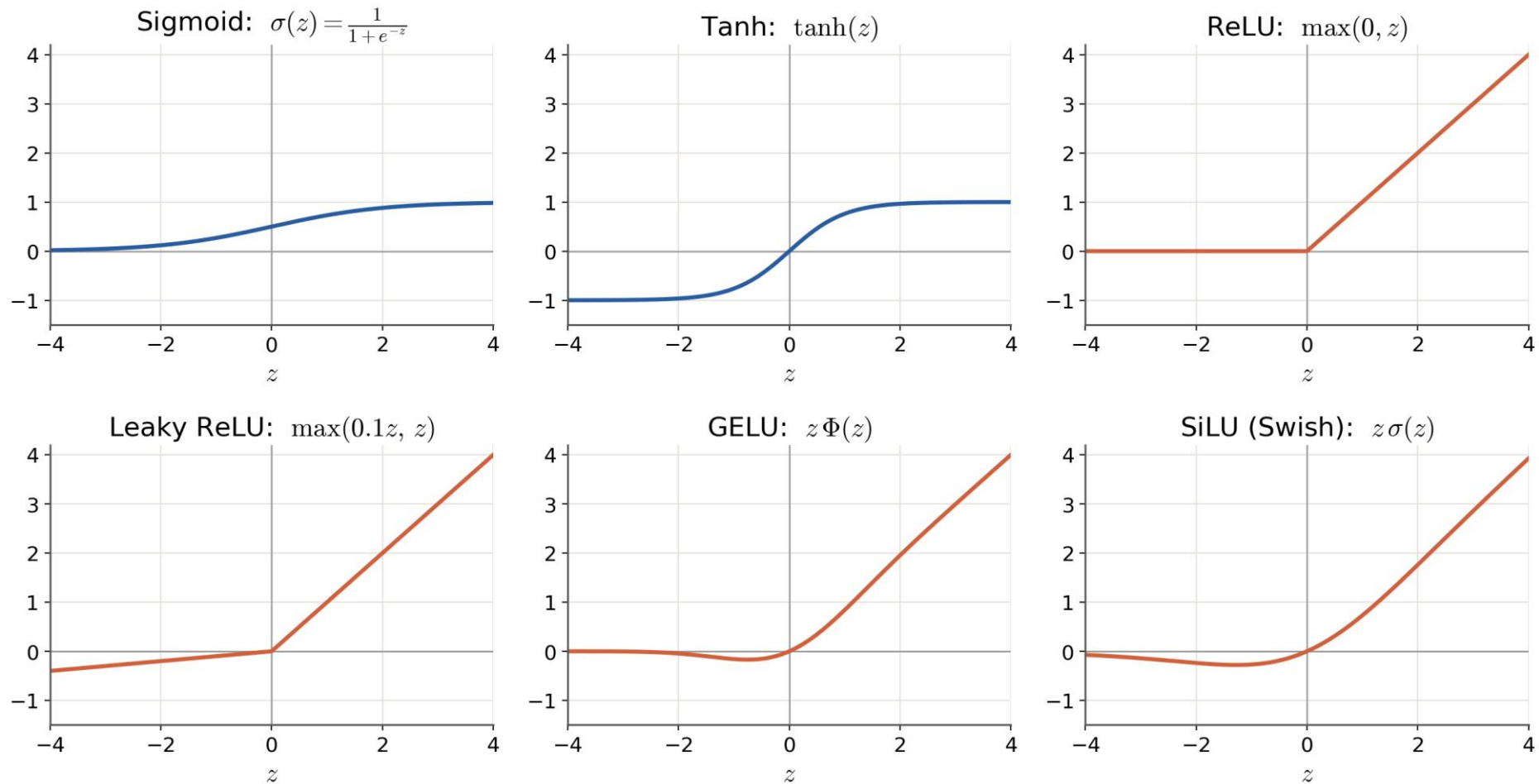


input: a "0"
 $\sum_j w_j x_j = -7.7$



- **Flatten** the image row by row into **784 inputs** $x_1, \dots, x_{784}$: pixel values from 0 to 1
- One neuron has **784 weights**, one per pixel, plus one **bias** b
- **Weights** (set by hand): **+1** for pixels in a band near the top, **-1** for pixels just above and below it, 0 elsewhere
- Reshaped to 28 x 28, the weights form a pattern: a detector for a horizontal bar at the top
- For the 7: the band adds 32.4 and the red rows subtract 11.5, so $\sum_j w_j x_j = 20.9$
- Bias $b = -10$ sets the threshold: only the 7 gives $z > 0$ and activates the neuron

Activation functions



Blue: squash z into a fixed range. **Orange:** gate z , passing positive values and blocking negative ones.

Choosing an activation

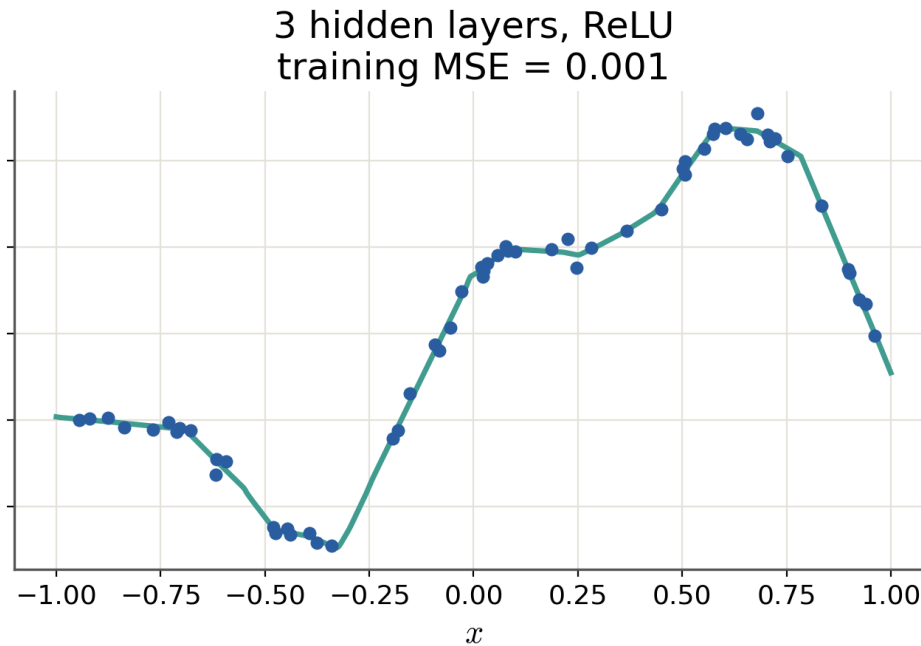
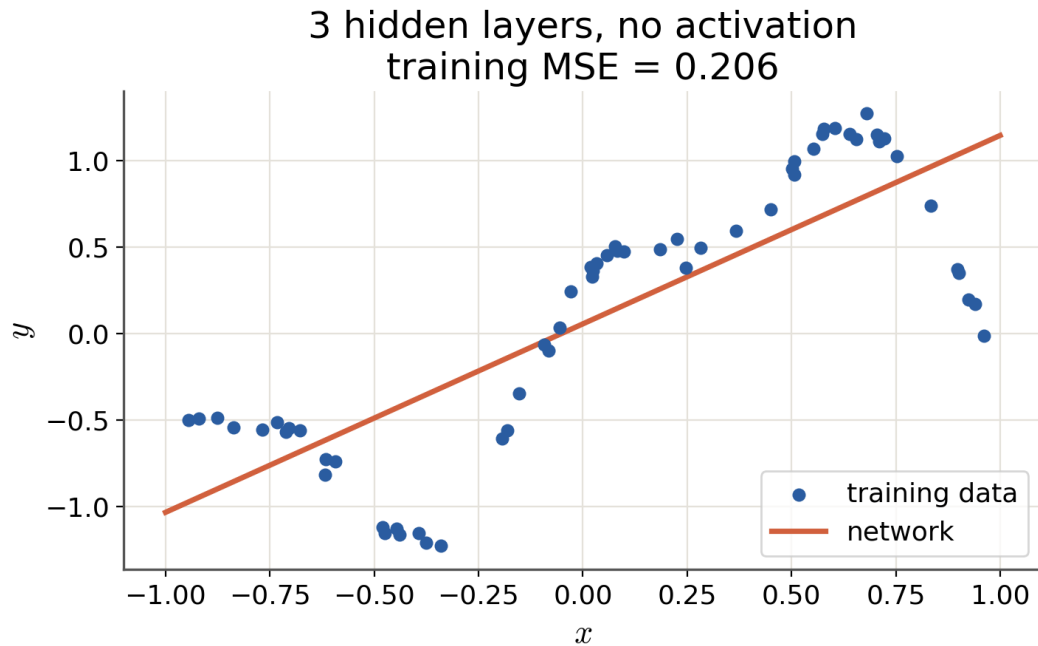
- **Sigmoid and tanh**
 - Smooth and bounded; sigmoid outputs lie in $(0, 1)$, tanh in $(-1, 1)$ and is centered at zero
 - Saturate for large $|z|$: the slope is nearly zero, so **gradients vanish in deep networks**
- **ReLU and leaky ReLU**
 - Cheap, and does not saturate for $z > 0$; the default choice for most networks
 - A unit with $z < 0$ for every input has zero gradient and stops learning (“**dead ReLU**”); leaky ReLU keeps a small slope
- **GELU and SiLU**
 - **Smooth versions of ReLU**; standard in transformers, and GELU is used in the Lecture 1 CNN
 - Smoothness matters when the loss contains derivatives of the network output (physics-informed losses)

Why the nonlinearity matters

Two layers **without** an activation:

$$\mathbf{a}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$
$$\hat{y} = W^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} = \underbrace{W^{(2)}W^{(1)}}_W \mathbf{x} + \underbrace{W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}}_b$$

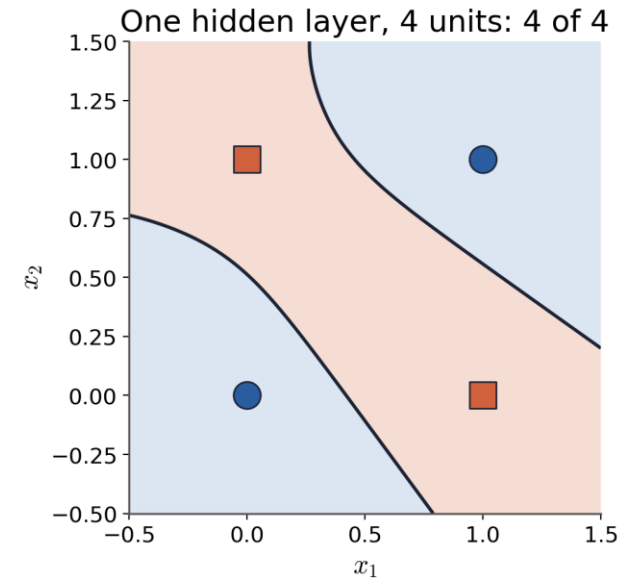
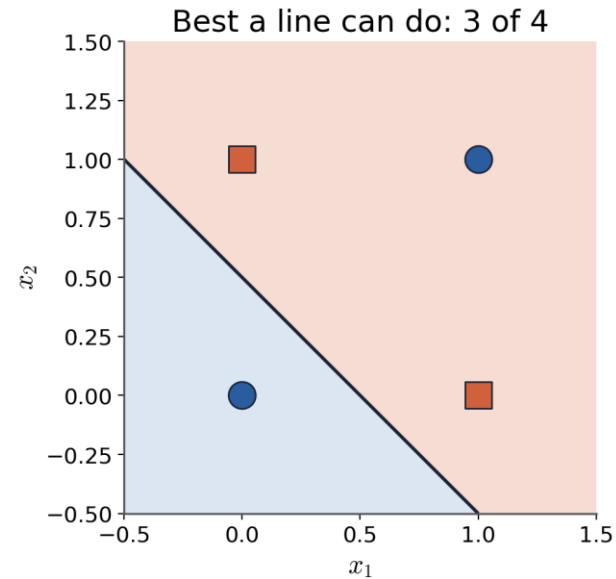
- Stacked linear layers collapse into one linear layer. Depth adds nothing without σ .



Example: XOR

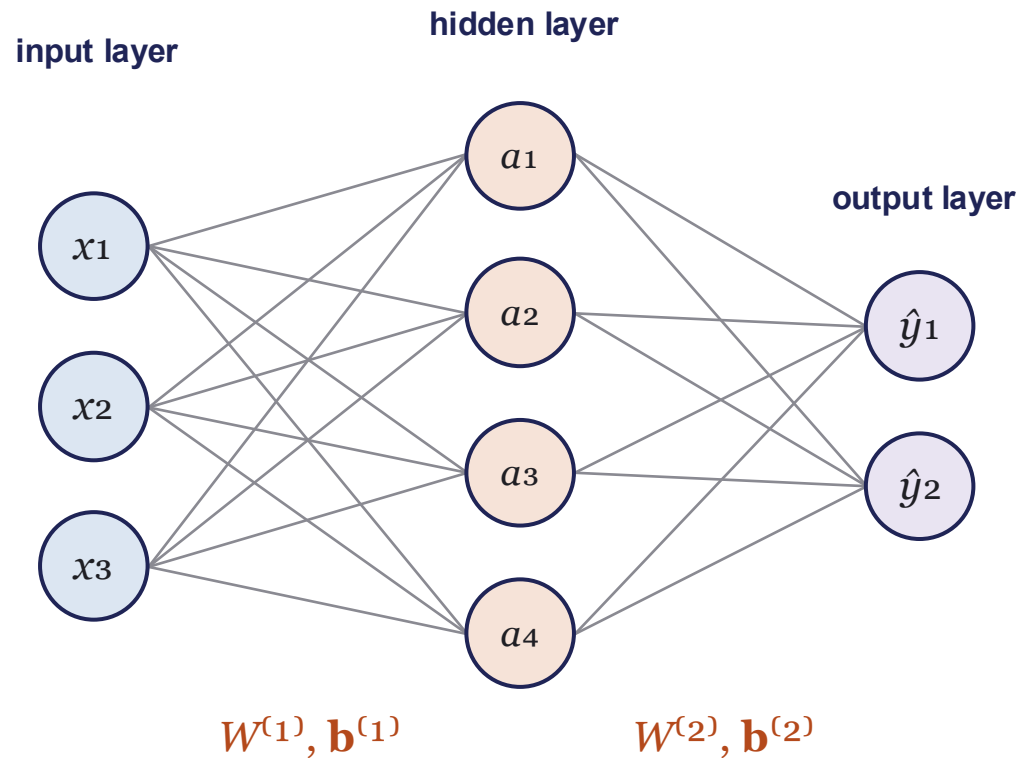
x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

1 when exactly one input is 1



- No straight line separates the two classes, so [a single perceptron fails](#) (Minsky & Papert, 1969)
- One hidden layer with a nonlinearity solves it: the hidden units compute new features in which the classes become separable

A shallow neural network

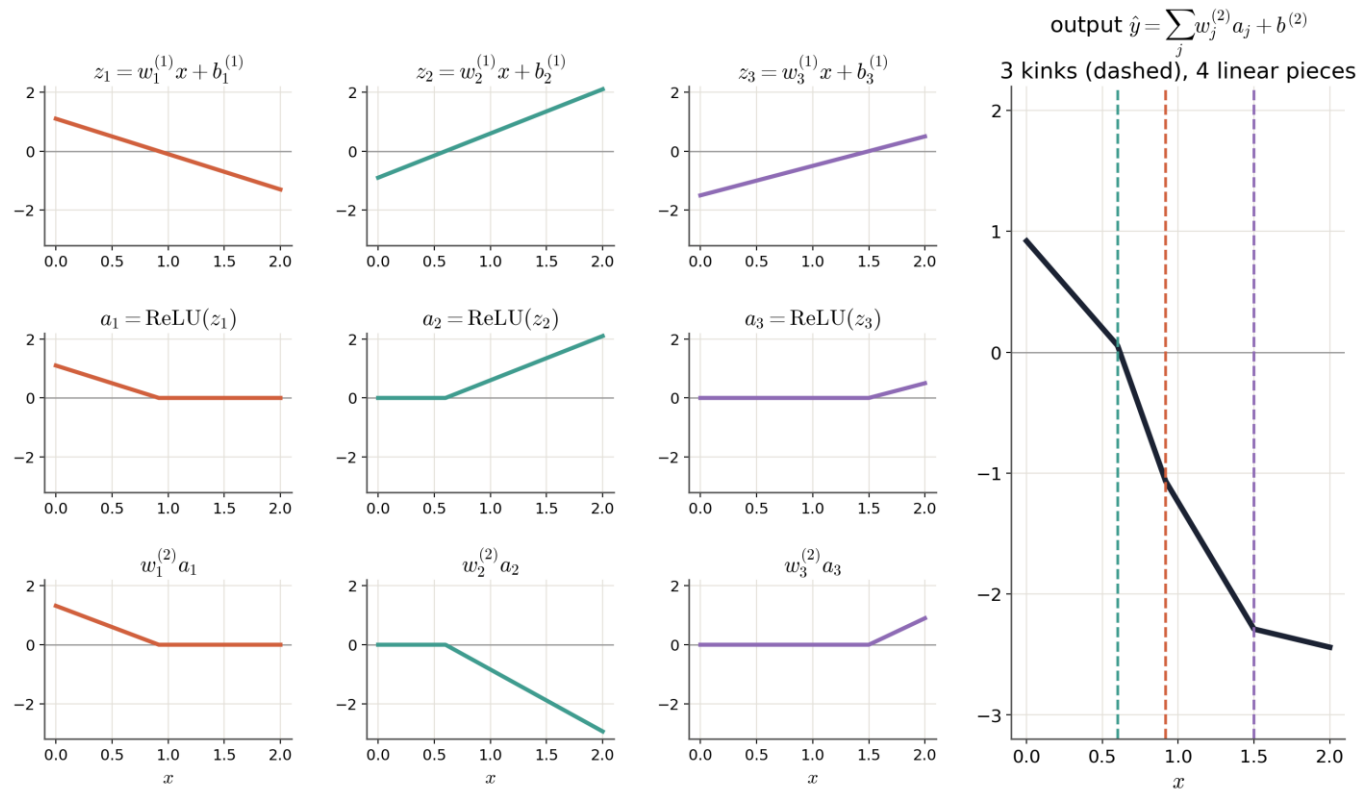


$$\mathbf{a}^{(1)} = \sigma(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{\mathbf{y}} = W^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

- Each line is a weight; each hidden and output neuron has a bias
- Superscript (l) = layer: $W^{(1)}$ has one row of weights per hidden neuron, $W^{(2)}$ one row per output
- **Width** = number of hidden neurons.
- Fully connected: every input feeds every hidden neuron
- Parameters: $3 \cdot 4 + 4 + 4 \cdot 2 + 2 = 26$

How a shallow ReLU network builds a function



- **Row 1:** each hidden neuron computes a line in x
- **Row 2:** ReLU clips it at zero: one kink per neuron
- **Row 3:** output weights scale each one
- **Right:** the sum is piecewise linear

With one input, N hidden neurons give at most $N + 1$ linear pieces. More neurons, more pieces, closer fit.

Universal approximation theorem

Theorem (Cybenko 1989; Hornik 1991; Leshno et al. 1993)

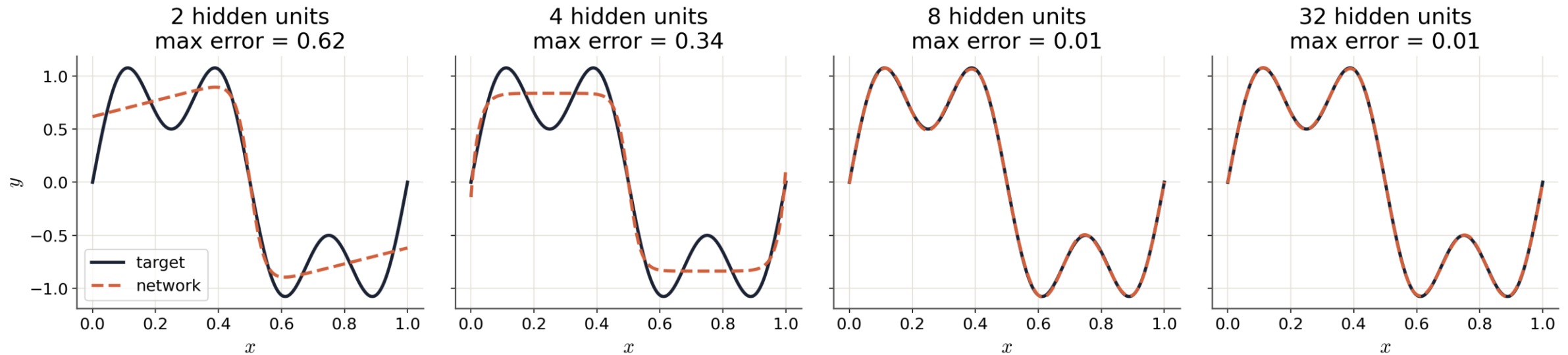
Let σ be continuous and not a polynomial. For every continuous function f on a compact set $K \subset \mathbb{R}^n$ and every $\varepsilon > 0$, there is a one hidden layer network G with some number N of hidden neurons such that

$$G(\mathbf{x}) = \sum_{j=1}^N w_j^{(2)} \sigma(\mathbf{w}_j^{(1)} \cdot \mathbf{x} + b_j^{(1)}), \quad \sup_{\mathbf{x} \in K} |f(\mathbf{x}) - G(\mathbf{x})| < \varepsilon$$

Same notation as before: $\mathbf{w}_j^{(1)}$, $b_j^{(1)}$ are hidden neuron j 's weights and bias; $w_j^{(2)}$ is its output weight.

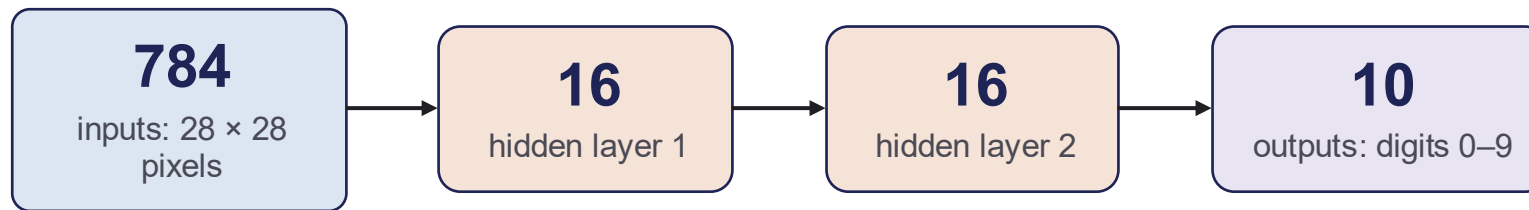
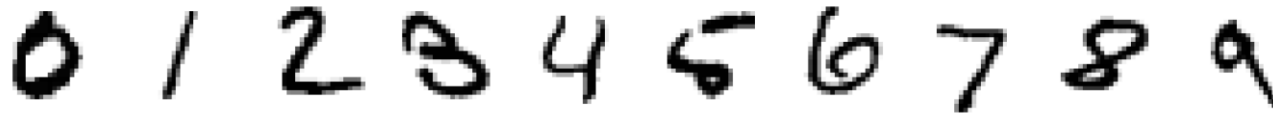
- **In words:** one hidden layer with enough neurons can approximate any continuous function to any accuracy
- **It does not say** how many neurons are needed, how to find the weights, or how much data is required

Universal approximation in practice



- **Target:** $f(x) = \sin(2\pi x) + 0.5 \sin(6\pi x)$, 200 samples on $[0, 1]$. One hidden layer of tanh neurons
- More neurons $\rightarrow$ smaller error. Each neuron adds a step the output can use
- Training does not always find the good network: shown is the best of 10 random initializations

A deep network: the MNIST “hello world”

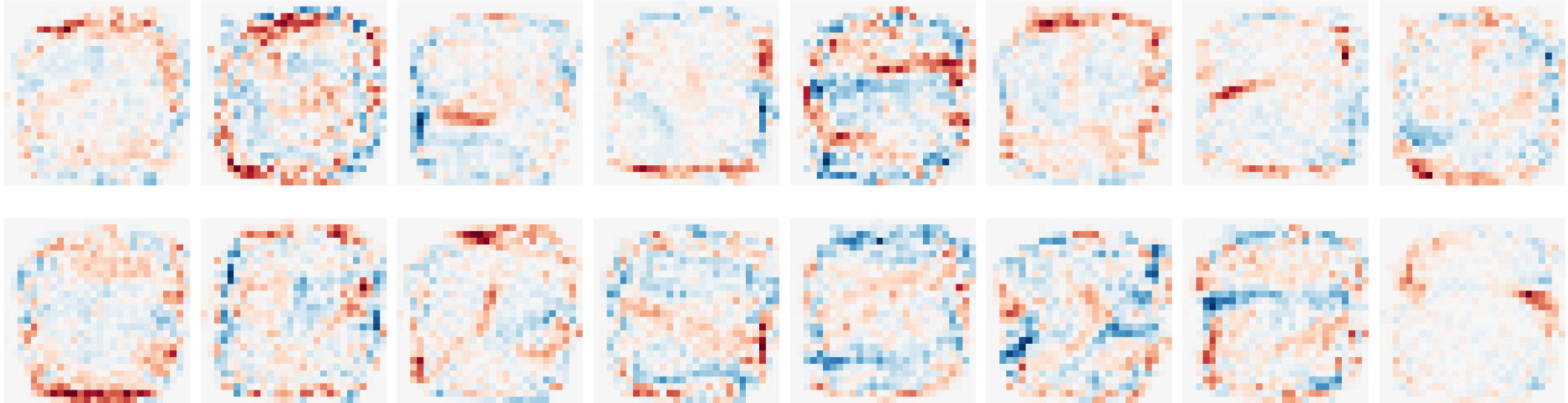


$$\mathbf{a}^{(0)} = \mathbf{x}, \quad \mathbf{a}^{(l)} = \sigma(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}), \quad l = 1, \dots, L$$

$$\underbrace{784 \cdot 16 + 16}_{\text{layer 1}} + \underbrace{16 \cdot 16 + 16}_{\text{layer 2}} + \underbrace{16 \cdot 10 + 10}_{\text{output}} = 13,002$$

- Each layer's activations are the next layer's input.
Depth L = number of layers
- Prediction = the output neuron with the largest activation
- Trained here (sigmoid hidden layers, softmax output, cross entropy loss, Adam, 40 epochs): **94.5%** correct on 10,000 test images

What did the first hidden layer learn?



- Each square: the 784 weights into one of the 16 neurons, drawn as an image (blue +, red -)
- The hope: layer 1 finds strokes, layer 2 combines them into loops and lines. The reality: mostly noisy patterns, yet 94.5% accuracy
- A network can work well without learning what we would call meaningful features. Understanding what a network actually learned is its own field (**mechanistic interpretability**)

Width vs. depth

- **Width**

- One hidden layer is enough in principle (universal approximation)
- But for general smooth functions the neurons needed grow exponentially with the input dimension

- **Depth**

- For compositional functions (built from simpler functions of few variables), depth can be exponentially more efficient than width
- These results are about what a network can represent, not what training finds. Deep networks are harder to train

- **In practice**

- At a fixed parameter count, total size matters more than shape. Width and depth are tuned on validation data

Loss functions

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Mean squared error (regression): large errors count much more than small ones

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

Mean absolute error (regression): less sensitive to outliers

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \hat{p}_{ic}$$

Cross-entropy (classification, e.g. MNIST): $\hat{p}$ is the predicted probability of each class

- For a field, MSE loss averages over samples, channels and grid points
- Every grid point counts equally, so a thin bubble interface adds little to the loss even when it is wrong

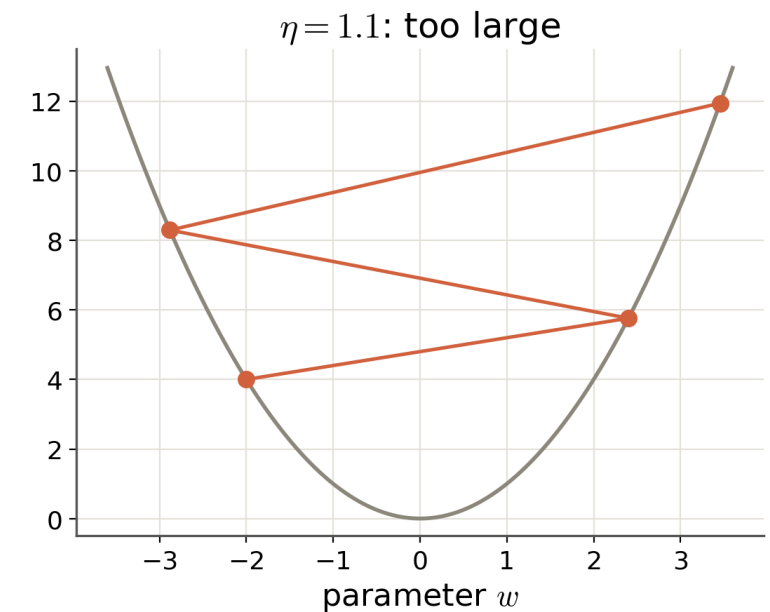
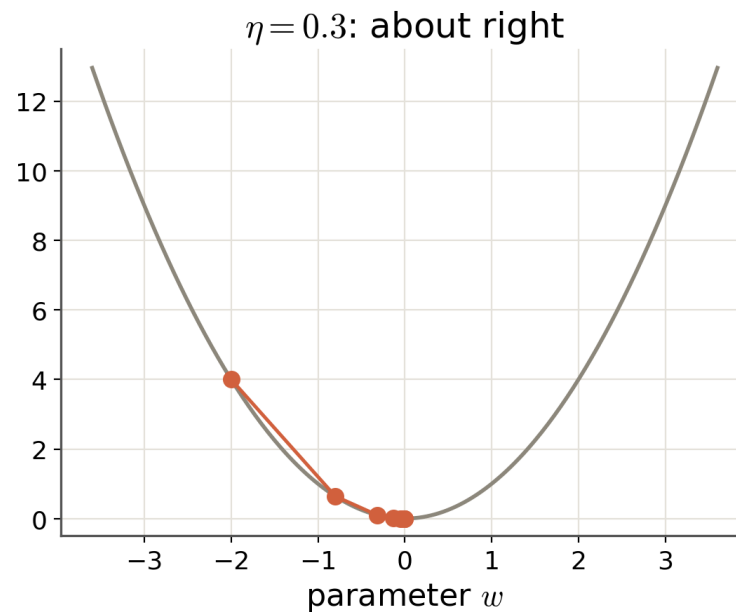
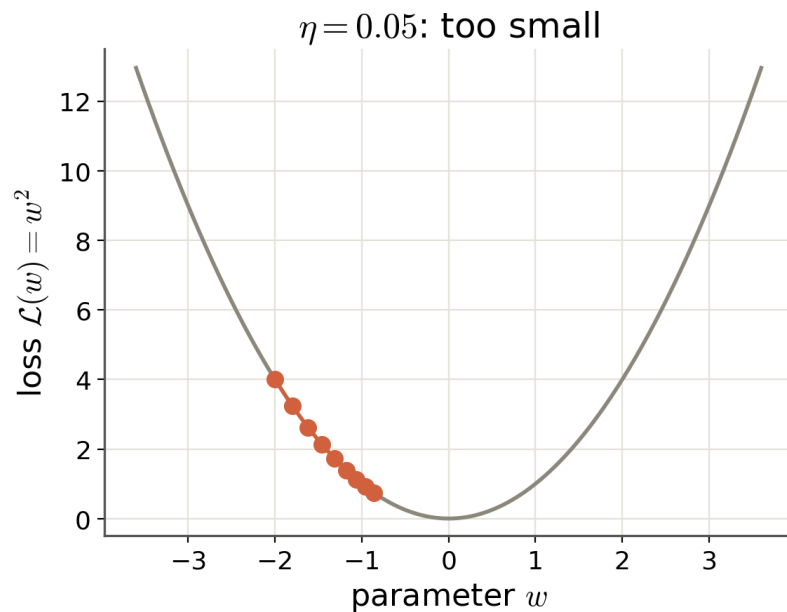
Training: how do we find 13,002 numbers?

- Training means minimizing the loss $\mathcal{L}(\theta)$ over all weights and biases: 13,002 for MNIST, millions for real models
- Linear regression has a closed-form minimum. A neural network does not
- Trying combinations is hopeless: 10 values per parameter is 10^{13002} networks
- What we can compute cheaply is the **gradient** $\nabla \mathcal{L}(\theta)$: the direction in which the loss increases fastest
- So start from random parameters and repeatedly take a small step downhill

Gradient descent: how to step. **Backpropagation:** how to compute the gradient.

Gradient descent

- $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$
- Step against the gradient; repeat until the loss stops decreasing
 - η is the **learning rate**, the step size

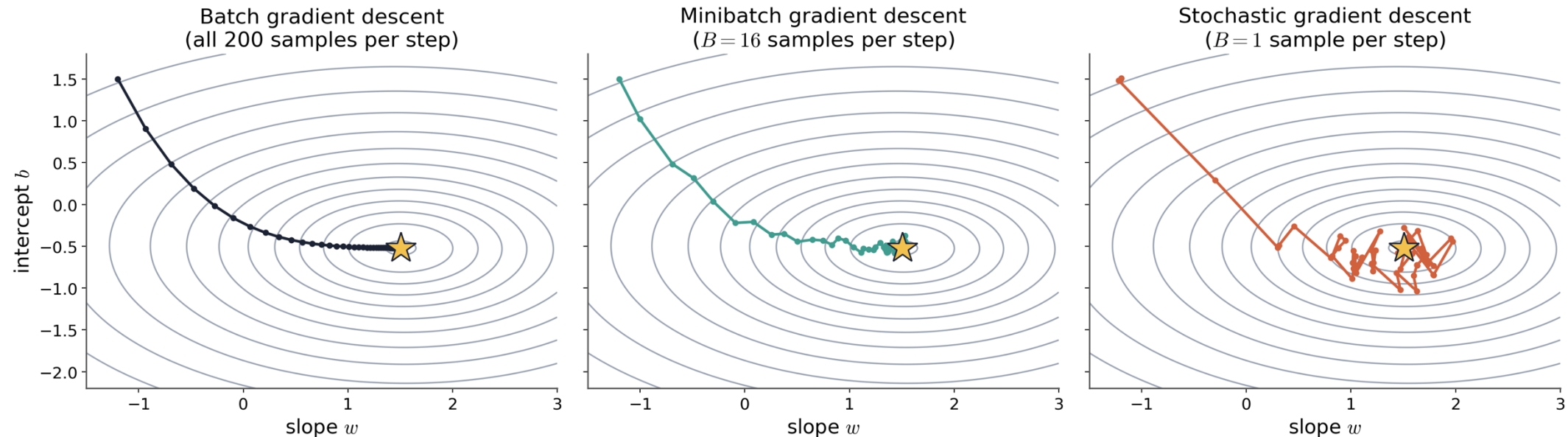


Too small: slow. Too large: overshoots and diverges. For $\mathcal{L} = w^2$, any $\eta > 1$ diverges.

Batch, minibatch and stochastic gradient descent

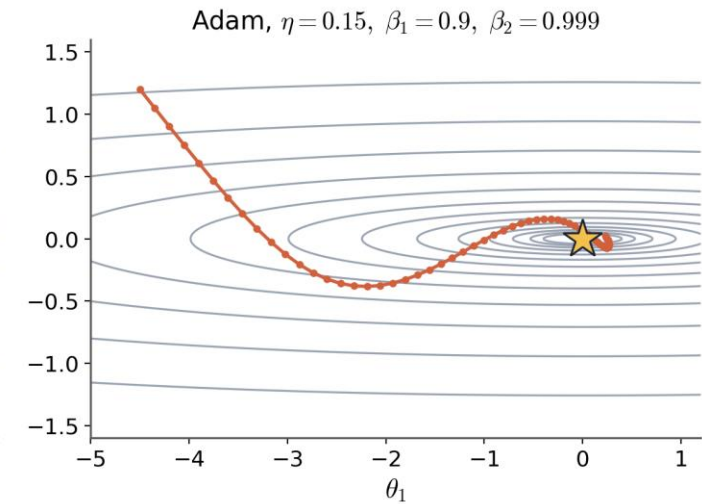
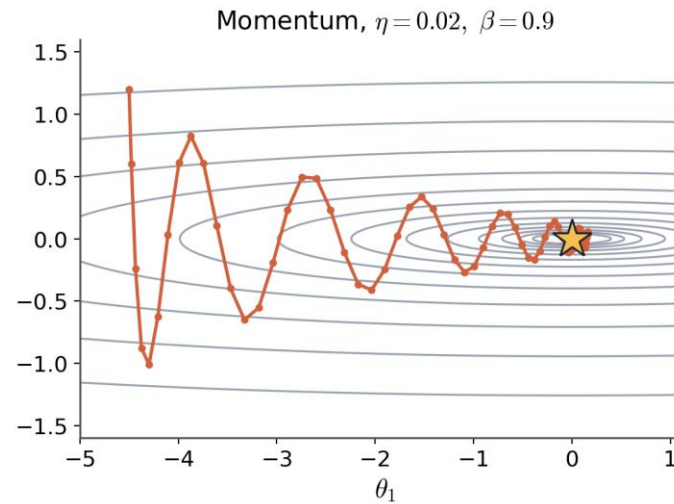
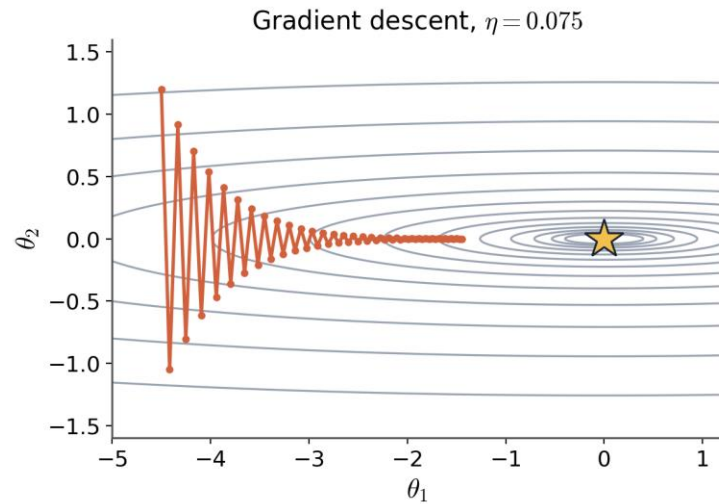
$$\nabla_{\theta} \mathcal{L}(\theta) \approx \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\theta} \ell_i(\theta)$$

- The full gradient needs every training sample. Estimate it from a random **minibatch** of B samples instead



- Smaller batches: cheaper, noisier steps. The noise can help escape poor regions of a non-convex loss
- **Step:** one update. **Epoch:** one pass through the data. MNIST: 50,000 images, $B = 64 \rightarrow 782$ steps per epoch

Momentum and Adam



Adam

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

Momentum

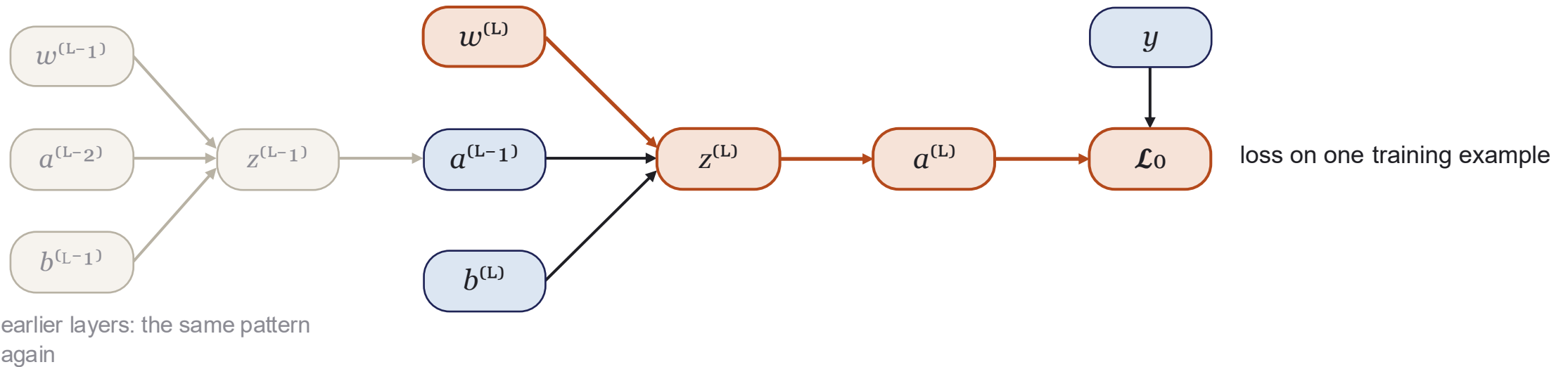
$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + \mathbf{g}_t$$

$$\theta_t = \theta_{t-1} - \eta \mathbf{m}_t$$

- $\mathbf{g}_t$ = gradient at step t
- Momentum accumulates past gradients: it speeds up along the valley and damps the zigzag
- Adam also divides by the size of recent gradients, so each parameter gets its own step size.

Backpropagation: start with one neuron per layer

$$z^{(L)} = w^{(L)}a^{(L-1)} + b^{(L)}, \quad a^{(L)} = \sigma(z^{(L)}), \quad \mathcal{L}_0 = (a^{(L)} - y)^2$$



- Each arrow: the quantity on the right is computed from the ones on the left
- A nudge to $w^{(L)}$ reaches the loss only through $z^{(L)}$ and $a^{(L)}$ (orange path), so its effect is a product of three derivatives

Backpropagation: the chain rule

$$\frac{\partial \mathcal{L}_0}{\partial w^{(L)}} = \frac{\partial z^{(L)}}{\partial w^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial \mathcal{L}_0}{\partial a^{(L)}} = \underbrace{a^{(L-1)}}_{\text{previous activation}} \underbrace{\sigma'(z^{(L)})}_{\text{slope of } \sigma} \underbrace{2(a^{(L)} - y)}_{\text{error}}$$

- A weight matters more when the neuron feeding it is active, when σ is steep, and when the output is far from the target

Bias ($\partial z / \partial b = 1$)

$$\frac{\partial \mathcal{L}_0}{\partial b^{(L)}} = 1 \cdot \sigma'(z^{(L)}) 2(a^{(L)} - y)$$

Previous activation ($\partial z / \partial a^{(L-1)} = w^{(L)}$)

$$\frac{\partial \mathcal{L}_0}{\partial a^{(L-1)}} = w^{(L)} \sigma'(z^{(L)}) 2(a^{(L)} - y)$$

- Knowing $\partial \mathcal{L}_0 / \partial a^{(L-1)}$, repeat the same step for layer $L - 1$, then $L - 2$, ...: the sensitivity is passed **backward**
- The full gradient averages these over the training examples (over a minibatch in SGD)

Backpropagation with many neurons per layer

$$z_j^{(l)} = \sum_k w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}, \quad a_j^{(l)} = \sigma(z_j^{(l)})$$

$$\frac{\partial \mathcal{L}_0}{\partial w_{jk}^{(l)}} = a_k^{(l-1)} \sigma'(z_j^{(l)}) \frac{\partial \mathcal{L}_0}{\partial a_j^{(l)}}$$

$$\frac{\partial \mathcal{L}_0}{\partial a_k^{(l-1)}} = \sum_j w_{jk}^{(l)} \sigma'(z_j^{(l)}) \frac{\partial \mathcal{L}_0}{\partial a_j^{(l)}}$$

- j indexes neurons in layer l , k in layer $l - 1$
- Each weight: the same three factors as before
- New: neuron k feeds every neuron in the next layer, so its sensitivity is a **sum** over j

- Start at the output and move back one layer at a time: one backward pass gives all 13,002 derivatives, at a cost of a few forward passes
- This is **reverse-mode automatic differentiation**. PyTorch records the computation in the forward pass; `loss.backward()` runs it in reverse

The training loop in PyTorch

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(epochs):
    for inputs, targets in loader:
        pred = model(inputs)
        loss = F.mse_loss(pred, targets)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

- loader yields minibatches
- Forward pass and loss
- Clear old gradients (PyTorch accumulates them)
- Backpropagation fills in $\nabla \mathcal{L}$
- Adam updates every parameter

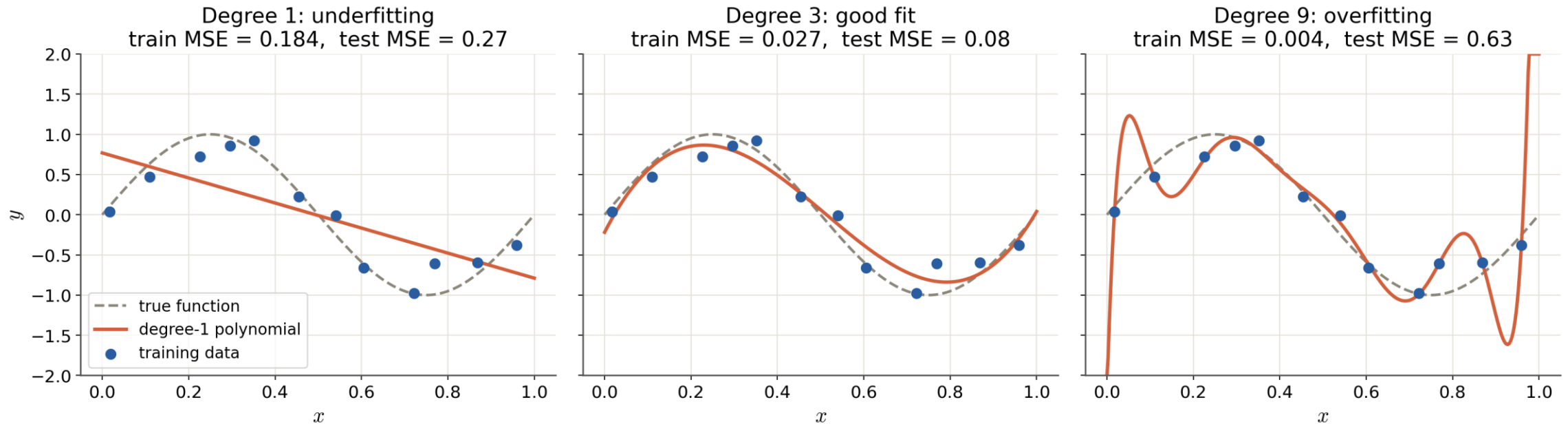
This is the `train()` function from the Lecture 1 notebook.

Training, validation and test sets

- **Training set**
 - Used to fit the parameters; the only data the optimizer sees
- **Validation set**
 - Used to choose **hyperparameters**: learning rate, width, depth, number of epochs, regularization strength
 - Never used to compute gradients
- **Test set**
 - Used once, at the end, to estimate performance on new data
 - Tuning on the test set turns it into a validation set, and the reported error becomes optimistic

How to split simulation data is not obvious. We return to this at the end of the lecture.

Underfitting and overfitting

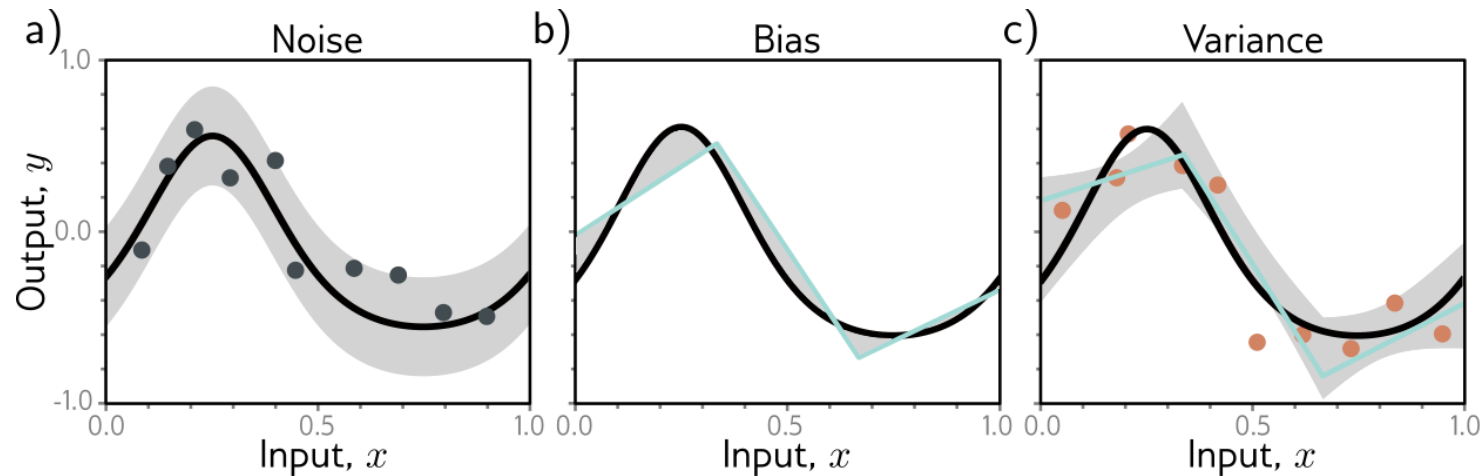


- **Underfitting:** the model is too simple; training and test error are both high
- **Overfitting:** the model fits the noise; training error is low, test error is high
- Training error always falls as the model grows. Only held-out data shows the difference

Bias-variance decomposition

Expected test error at x , averaged over training sets drawn from the same distribution:

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{bias}^2} + \underbrace{\mathbb{E} \left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2 \right]}_{\text{variance}} + \underbrace{\sigma_\epsilon^2}_{\text{noise}}$$

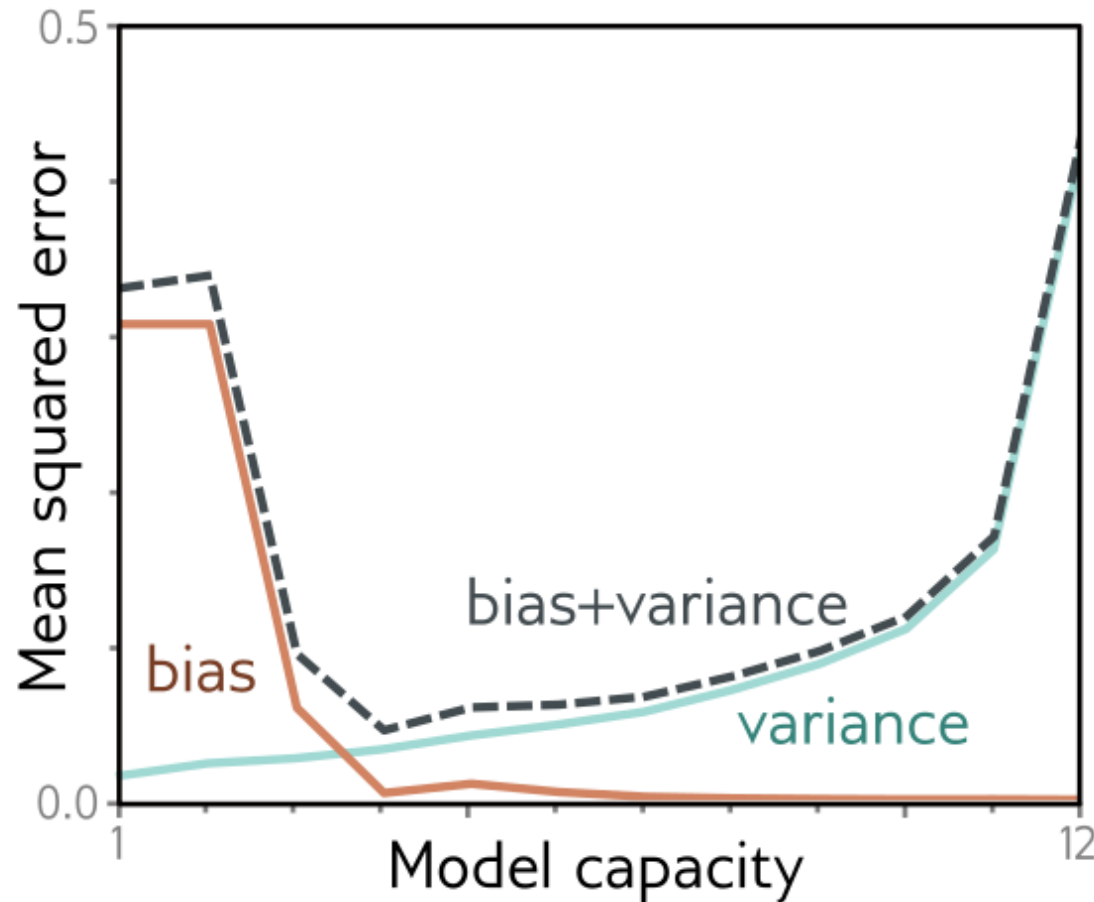


Noise: randomness in the data itself; no model can remove it

Bias: the model is systematically wrong, even on average (too inflexible)

Variance: the fit changes a lot with the particular training set (too flexible for the data)

Bias-variance trade-off



- More capacity lowers bias and raises variance
- More training data lowers variance, so larger models become usable
- The best capacity balances the two, and is chosen on the validation set
- Caveat: very large networks can generalize well again past the point where they fit the training data exactly (“double descent”)

Regularization: why and when

- **What it is**
 - Anything that discourages the model from fitting the noise in the training data
 - It accepts a little more bias in exchange for a lot less variance
- **When you need it**
 - Validation loss stops falling or rises while training loss keeps falling
 - Many more parameters than independent training samples, which is common with simulation data
 - Noisy data, e.g. experimental measurements
- **The options**
 - Penalize large weights (L2, L1), stop training early, drop units at random, augment the data, or constrain the architecture

L2 and L1 regularization

Add a penalty on the weights to the loss:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda R(\theta)$$

- **L2 (weight decay, ridge)**

- Each step shrinks every weight toward zero in proportion to its size
- Weights become small but rarely exactly zero; the fitted function is smoother

$$R_{L2}(\theta) = \sum_i w_i^2, \quad \frac{\partial R_{L2}}{\partial w_i} = 2w_i$$

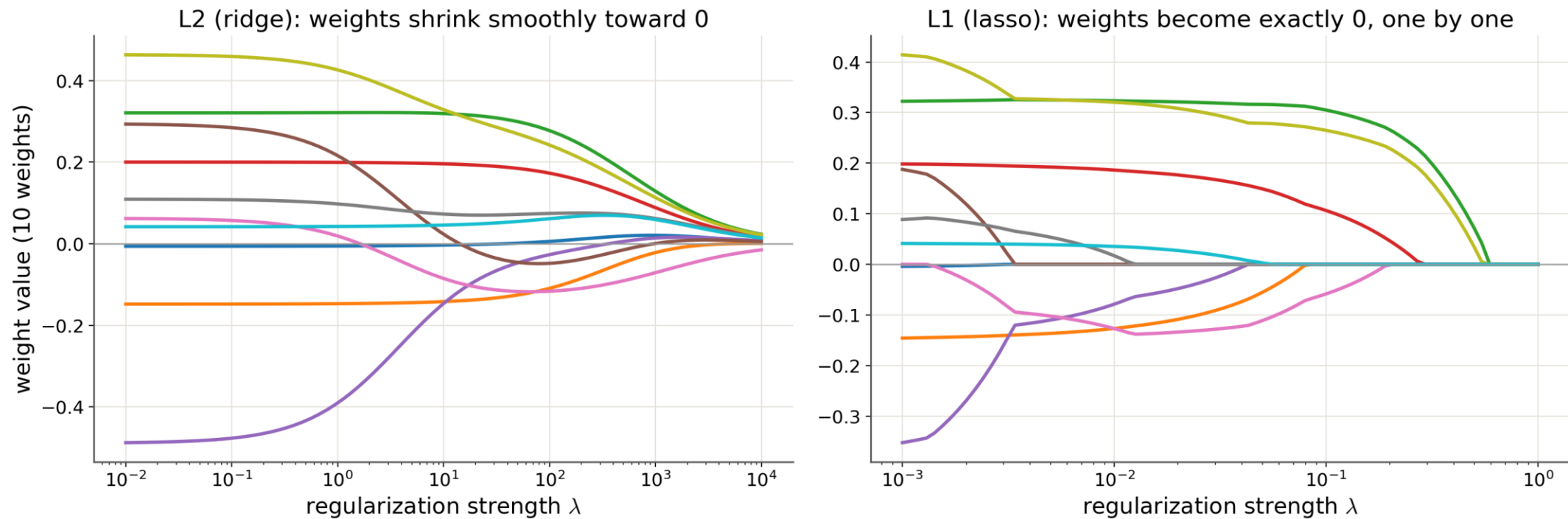
- **L1 (lasso)**

- Each step pushes weights toward zero by a constant amount
- Small weights become exactly zero: a **sparse** model

$$R_{L1}(\theta) = \sum_i |w_i|, \quad \frac{\partial R_{L1}}{\partial w_i} = \text{sign}(w_i)$$

λ sets the strength: too small still overfits, too large underfits. Choose it on the validation set.

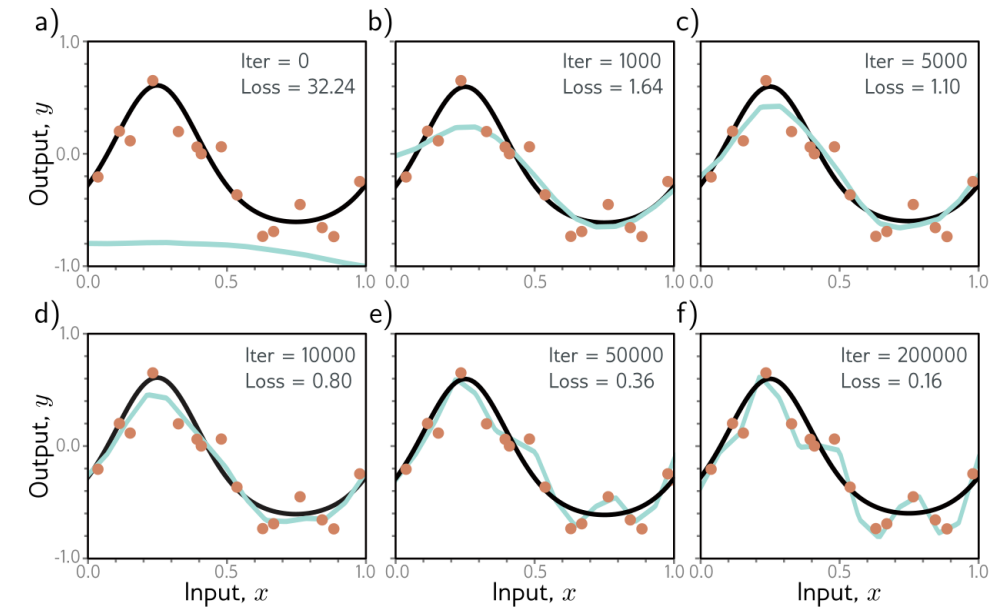
L2 vs. L1: what happens to the weights



- Each line is one of 10 weights, refit at each λ . Left to right: stronger regularization
- L1 selects inputs: at moderate λ only a few weights are nonzero. L2 keeps all of them, just smaller

Other ways to regularize

- **Early stopping:** keep the weights from the epoch with the lowest validation loss
- **Dropout:** randomly zero a fraction of hidden units at each training step, so no unit can rely on another
- **Data augmentation:** add transformed copies of the training data. For physics: rotations, reflections and other symmetries of the equations
- **Architecture:** restrict what the model can represent. A CNN is an MLP with locality and weight sharing imposed



Early stopping: the fit gets more complex the longer it trains

In scientific ML, the strongest regularizer is usually physical structure built into the architecture or the loss.

Physical fields have different scales

Field	Min	Max	Std
temperature	0.00	0.99	0.10
velx	-7.1	8.0	0.40
vely	-4.7	5.9	0.54
pressure	-155	141	24.1
dfun	-12.2	1.3	2.6

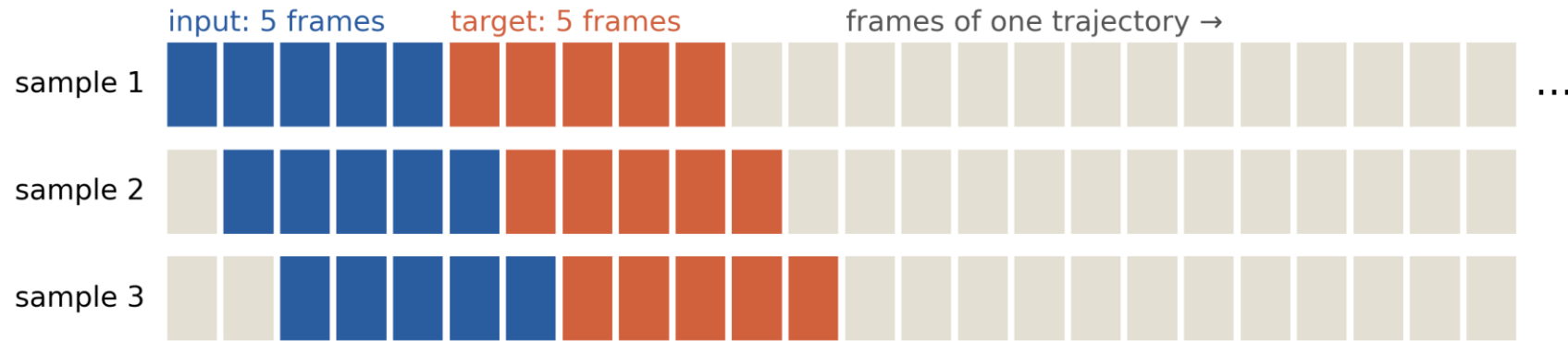
BubbleML, T_{wall} = 100 °C, all 201 frames (nondimensional units)

- Unscaled, pressure dominates the MSE and its gradients; temperature barely counts
- Standardize each channel:

$$\tilde{x} = \frac{x - \mu}{\sigma}$$

- Compute μ and σ from the **training set only**

One trajectory, many correlated samples



- T frames with k inputs and m targets give $T - k - m + 1$ samples. BubbleML: 201 frames, $5 \rightarrow 5$, gives 192 per simulation
- These samples share initial condition, parameters and most of their frames. They are **not independent**
- **Split by what you want to generalize over:** held-out trajectories (new initial conditions), held-out parameters (new T_{wall} , as in Lecture 1), or held-out time (forecasting)
- A random split of frames can leak (depends on the sampling interval)

Generalization and evaluation

- **Interpolation vs. extrapolation**

- Train on $T_{\text{wall}} = 100$ and 106 °C, test on 103 °C: interpolation. Testing at 109 °C would be extrapolation
- The shift can be in the state too: Lecture 1, Exercise 3 trains on modes up to $K = 3$ and tests on $K = 6$
- A network has no built-in reason to be right outside its training data

- **The training loss is not the evaluation metric**

- Trained on one-step MSE in normalized units; evaluated on relative error over a rollout, in physical units:

$$\text{relative } L_2 \text{ error} = \frac{\|\hat{u} - u\|_2}{\|u\|_2}$$

- Small one-step error does not guarantee a stable rollout (Lecture 1 plots); averages hide local failures such as a smeared interface

Summary

- A **neuron** is a weighted sum plus a bias, passed through a nonlinearity. Without the nonlinearity, depth adds nothing
- One hidden layer can approximate any continuous function; depth can be far more efficient for compositional functions, but is harder to train
- **Training** = minimizing a loss with minibatch gradient steps; backpropagation gets every gradient in one backward pass
- Test error = $\text{bias}^2 + \text{variance} + \text{noise}$. Validation data and regularization keep the balance
- Simulation data breaks the usual assumptions: mixed scales, correlated samples, new conditions at test time

AFTER CLASS

If today's refresher went fast for you

- **3Blue1Brown, *Neural networks*, Chapters 1–4 (3blue1brown.com)**
 - Same MNIST network and notation as today. Ch. 1: neurons and layers · Ch. 2: gradient descent · Ch. 3-4: backpropagation
- **Prince, *Understanding Deep Learning* (free PDF, udlbook.github.io)**
 - Ch. 2-4: supervised learning, shallow and deep networks · Ch. 6-7: training and gradients · Ch. 8-9: generalization and regularization
- **Optional: Nielsen, *Neural Networks and Deep Learning*, Ch. 4**
 - An interactive, visual proof of universal approximation

Next, Lecture 3 (Thu, Oct 1): PDE and numerical fundamentals, where simulation data comes from